



COMP2000: Semester Project

Overview

Your task is to make a simulation of some natural or imagined system (ecosystem, traffic, physics, etc) where entities interact and produce emergent behaviour. You will find many existing examples of such systems such as:

- predator-prey systems
- Conway's game of life
- traffic simulation — cars, traffic lights, congestion that builds and clears
- flocking / boids — birds or fish that self-organise with simple rules
- epidemic spread — agents move, infect, recover, with Ro visibly emergent
- ant colony foraging — individual ants leave pheromone trails, paths emerge
- forest fire — trees grow, lightning strikes, fire spreads with wind
- market simulation — buyers and sellers set prices, boom/bust cycles emerge

Conway's game of life is such great fun so it is a good place to start.

You will need to create such a system, but that is not the overall goal. The overall goal is to *use the creation of such a system to understand and communicate OO programming*.

It is up to your team to decide on a simulation to attempt and you will work on it together with your team throughout semester. Your submissions will be *individual*. You will take the work from your team, add your own reflections on the code, and submit that. This type of hybrid team/individual assessment allows you to get the benefits of working in a team without the downsides of relying on others for your grades.

You will see two worksheets to complete, one in each of the submission links, and two rubrics which show you the answers we are looking for where you can show off your understanding.

Your simulation is the *canvas*, not the *art*. The worksheets (worksheet-7.md and worksheet-13.md) are where you show what you understand. Each question asks you to point to a place in your code and explain *why* you designed it that way — why you chose an inheritance hierarchy over an interface, where generics keep your collections safe, which design pattern solved a growth problem, and how lambdas or streams made a tangled loop disappear.

By Week 13 your simulation will be genuinely interesting to run, but the grade comes from how clearly you can discuss the design decisions behind it.

Your simulation needs to be watchable. A simulation you can't see is a simulation you can't understand. So in addition to the logic, you must build an interface that shows your simulation in action. A simple graphical display that draws your entities as they move and interact is enough. Do not spend time on a polished GUI: Swing or JavaFX will eat your semester and add nothing to your design. Just draw on a `JPanel` inside a `JFrame`, redrawing a few times a second, and you are done. Plan for a window early in the semester rather than a last-week afterthought, and remember that the Week 13 showcase expects to see your simulation running.



Rubrics

Note

Because this is an individual submission, the github url you provide your marker *cannot be your team's github url*. You are responsible for setting up your own repository to link to. The repository can be a fork of your team's repository, it does not need to be fresh, but it must have its own url even if all the code is identical to the team's.

Week 7 Submission

Student submits exactly two files (do not zip them): the completed worksheet and your log book. The worksheet includes a link to your own public GitHub repository. Your URL must be your own fork of your team's repository — not your team's URL — because your submission is individual.

Note: it is an assumption that the submission will compile and run. Compilation failures attract zero grades.

item	F (0)	P (60)	CR (72)	D(84)	HD (100)
version control (20%)	no repo link or team's repo link	public GitHub repo link provided	student has regular commits across the submission period	commits are meaningful, each describes a discrete change; student can explain their commit history	repository includes branches, PRs, or equivalent collaborative workflow in their own contributions
program design (20%)	none	reflective report includes a basic class sketch	design identifies classes, fields, and their relationships	design uses inheritance hierarchy appropriately	design is accurate to the submitted code
generics and exceptions (20%)	weak or no entries for these weeks	log book entries present for the puzzle and break the code	entries explain how generics and exceptions work	discusses the role (or intentional absence) of each in own project	code correctly uses (or deliberately avoids) these features
uniqueness and creativity (20%)	a direct submission of in-class work	minor modifications to in-class work	some modifications to in-class work	substantial improvements to in-class work	unexpected or advanced design feature(s)
log book (20%)	no log book submitted	entries present for each activity	most entries signed (timely)	entries show reflection connecting the activity to the project	entries demonstrate insight and learning across multiple weeks

**Week 13 Submission**

Student submits exactly two files (do not zip them): the completed worksheet and your log book. The worksheet includes the URL of your own fork of your team's repository, where your final code lives. Your submission is individual, so the URL must be your own fork — not your team's URL.

Note: it is an assumption that the submission will compile and run. Compilation failures attract zero grades.

item	F(0)	P(60)	CR(72)	D(84)	HD(100)
design patterns (30%)	no pattern attempted	attempts to implement a pattern	pattern is implemented correctly	pattern is well-integrated into the existing design	pattern choice is justified; alternative patterns were considered
streams / lambdas / parallelism (30%)	none attempted	attempts to use at least one of these features	feature works correctly in the code	feature is well-integrated across the codebase	sophisticated use (e.g. parallel streams with thread safety, custom collector, lambda replacing a class hierarchy)
uniqueness and creativity (20%)	a direct submission of in-class work	minor modifications to in-class work	substantial improvements to in-class work	project exhibits emergent behaviour, non-trivial features, or impressive complexity	genuinely surprising or interesting outcome demonstrating deep engagement
log book + final reflection (20%)	no log book submitted	entries present for most weeks	entries present for all weeks, timely	entries connect activities to the project throughout the semester	final reflection critically evaluates what was learned, what trade-offs were made, and what could be improved